
SOFTWARE ARCHITECTURE BLUEPRINT · REVERSE-ENGINEERED

Necoyoad

*An extensive, diagram-rich reconstruction of a legacy
PHP/MySQL multi-store e-commerce and content
platform*

artifact: necoyoad_db.sql (76,467 bytes)
dump time: 2023-02-03 07:37 UTC
stack: PHP 8.1.2 / MySQL 5.7.17 / MyISAM
tables: 87 · diagrams: 18 · pages: 34

Reverse-Engineered Architecture Report

Based on the public database dump at
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — A Reverse-Engineered Software Architecture Blueprint

*An extensive, diagram-rich reconstruction of a legacy PHP/MySQL
multi-store e-commerce and content platform*

Abstract

This document is a verbose, deeply detailed blueprint of the **Necoyoad** web application, a custom PHP/MySQL platform that the author (*yosietserga*) published as a single phpMyAdmin database dump on GitHub in February 2023. The repository under analysis contains no application source code; only the `necoyoad_db.sql` dump is present. We therefore reconstruct the runtime architecture, the domain model, the cross-cutting patterns and the inferred user-facing flows by treating the 87-table relational schema as the primary, and largely sufficient, source of truth.

The reconstruction shows a system whose design vocabulary is clearly inherited from the OpenCart 1.x/2.x lineage (table names, the `product_to_category` junction style, the `url_alias` SEO layer, the `order_total` extension point, the `customer_group` pricing model) but extended far beyond a vanilla store with: (i) a polymorphic object graph (`object`, `object_to_category`, `object_to_store`, `description`, `property`, `status`, `url_alias`) that acts as a typed Entity-Attribute-Value (EAV) spine; (ii) a full content-management surface (`post`, `menu`, `banner`, `widget`, `theme`, `template`, `theme_style`); (iii) a marketing/CRM surface (`newsletter`, `campaign`, `contact`, `contact_list`, `campaign_contact`, `campaign_link`, `campaign_stat`, `campaign_link_stat`); (iv) a Venezuelan-style bank-transfer payment surface (`bank`, `bank_account`, `order_payment`, `balance`); (v) a warehouse inventory surface (`warehouse_movement`); and (vi) a self-hosted task scheduler (`task`, `task_queue`, `task_exec`) that drives the marketing campaigns.

We deliberately *do not* improve, modernise or recommend changes. The goal of the paper is purely descriptive: a faithful, exhaustive map of the system as it was the morning of 3 February 2023, the timestamp frozen into the dump header. Eighteen TikZ diagrams, six inventory tables and an appendix listing every one of the 87 tables accompany the prose.

Keywords: Necoyoad, OpenCart lineage, PHP 8.1, MySQL 5.7, MyISAM, phpMyAdmin dump, EAV, polymorphic associations, multi-store, multi-language, MVC, front controller, task scheduler, bank-transfer payments, CMS, marketing campaigns, SEO URL aliases.

Contents

1	Introduction	5
1.1	Subject of the Blueprint	5
1.2	Scope and Stance	5
1.3	Document Roadmap	5
2	Provenance and Artifact Description	6
2.1	The Repository	6
2.2	The Dump File Header	6
2.3	What the Header Tells Us	6
2.4	Database and Table Prefix	7
2.5	Engine, Charset and Collation Choices	7
2.6	The 87-Table Inventory at a Glance	7
3	Inferred Runtime Stack	7
3.1	The Apache-or-Nginx / PHP / MySQL Triangle	7
3.2	Why the Filesystem Still Matters	8
3.3	Cron or Pseudo-Cron	8
4	Architectural Style and Cross-Cutting Patterns	9
4.1	Style: OpenCart-derived MVC	9
4.2	Layered View	10
4.3	Four Cross-Cutting Patterns	10
4.3.1	Pattern 1 — Polymorphic Object Graph	10
4.3.2	Pattern 2 — EAV-style Descriptions	10
4.3.3	Pattern 3 — Multi-Store Scoping	11
4.3.4	Pattern 4 — Multi-Language Scoping	11
4.4	Status Convention	11
5	Schema Walkthrough by Domain	11
5.1	Domain 1 — The Polymorphic Object Spine	11
5.2	Domain 2 — Catalog and Products	12
5.2.1	Notable Columns in <code>product</code>	13
5.2.2	The Three Pricing Tiers	13
5.2.3	Product Attributes vs. Product Options	14
5.2.4	Tags and Taxonomy	14
5.3	Domain 3 — Customers, Addresses and Reviews	14
5.3.1	Notable Columns in <code>customer</code>	14
5.3.2	The Wallet: <code>balance</code>	15
5.3.3	Reviews and Likes	15
5.4	Domain 4 — Orders, Coupons and Checkout	15
5.4.1	The Snapshot Pattern in <code>order</code>	15
5.4.2	Order Status and History	15
5.4.3	<code>order_total</code> — The Extension Point	17
5.4.4	<code>order_payment</code> — Manual Bank Transfers	17
5.4.5	Coupons	17
5.5	Domain 5 — Banking and Wallet	17
5.6	Domain 6 — CMS, Posts, Banners, Menus	18
5.6.1	Posts	18
5.6.2	Banners	18
5.6.3	Menus	18

5.7	Domain 7 — Themes, Templates and Widgets	19
5.7.1	The Template Versioning Column	19
5.7.2	Widgets	19
5.8	Domain 8 — Marketing and Campaigns	19
5.8.1	The Campaign Lifecycle	19
5.8.2	The Tracking Schema	20
5.9	Domain 9 — Inventory and Warehouse	20
5.10	Domain 10 — Tax and Geography	21
5.11	Domain 11 — Users, Groups and Audit	21
5.12	Domain 12 — Tasks and Scheduler	22
5.13	Domain 13 — Localisation	22
5.13.1	Language	23
5.13.2	Currency	23
5.13.3	url_alias — The SEO Layer	23
5.14	Domain 14 — Notifications, Stats and Search	23
5.15	Domain 15 — Settings and Extensions	24
5.15.1	The Flat Settings Store	24
5.15.2	Extensions	24
6	Cross-Cutting Patterns, Formalised	24
6.1	Pattern Map	24
6.2	Polymorphic Association Inventory	24
6.3	The “params” / “settings” Escape Hatch	25
6.4	Composite Conventions	25
7	Inferred HTTP Request Lifecycle	26
7.1	Phase Notes	26
8	Inferred Customer Checkout Flow	26
8.1	Snapshot Writes at Confirmation	26
9	Inferred Task Scheduler Lifecycle	27
9.1	Real Cron vs. Pseudo-Cron	27
10	SEO and URL Routing	28
11	Security and Permissions Model	28
11.1	Layer 1 — Customer Authentication	28
11.2	Layer 2 — Admin Authentication	29
11.3	Layer 3 — Route-Level Authorisation	29
11.4	Audit Trail	29
12	Data Integrity Observations	29
12.1	Consequences of MyISAM	29
12.2	Consequences of the Polymorphic Pattern	29
12.3	Consequences of the “params” TEXT Escape Hatch	29
12.4	Consequences of the Snapshot Pattern	30
13	Closing Observations	30
13.1	The System Has Its Own Identity	30
13.2	The Venezuelan Context is Visible	30
13.3	The Schema Records Its Own Evolution	30
13.4	The Marketing Subsystem is the Most Sophisticated Part	30

13.5 The Audit Subsystem is Pervasive	30
14 Conclusion	31
A Full Table Inventory	31
B Polymorphic Association Inventory	32
C Common Column Conventions	33
D Engine, Charset and Collation Summary	34
E Glossary	34

1 Introduction

1.1 Subject of the Blueprint

Necoyoad is the public-facing name of a custom web application whose only publicly-published artifact is a phpMyAdmin database dump committed to GitHub by the user `yosietserga` under the repository github.com/yosietserga/necoyoad. The repository is, in the strictest sense, a single-file repository: it contains exactly one commit (“Add files via upload”), one branch (`main`), and one file — `necoyoad_db.sql`. There is no `README`, no license file, no `composer.json`, no `.htaccess`, no PHP source whatsoever. The application code itself is, at the time of writing, not publicly available.

This paper treats the SQL dump as a forensic snapshot of the application’s data layer and reconstructs, from that snapshot, the surrounding architecture that must have produced it. Database schemas carry a surprisingly large amount of architectural intent: every foreign-key column declares a dependency, every `enum` or `status` column declares a state machine, every `date_added` / `date_modified` pair declares an audit expectation, every polymorphic pair (`object_id`, `object_type`) declares an indirection point. Eighty-seven tables, taken together, are more than enough to reverse-engineer a coherent picture of the runtime.

1.2 Scope and Stance

The reader should treat this document as an *architecture blueprint*, not a tutorial, a review, or a refactoring proposal. Three rules govern the text below:

1. **Descriptive, not prescriptive.** We document what the system *is*. We do not propose changes, modernisations, or “best-practice” rewrites. Where a design choice looks unusual from a 2023-vantage point (for example, the use of the MyISAM engine, or the absence of declared foreign keys), we describe the consequences and leave it there.
2. **Inference clearly labelled.** Where the schema directly tells us something, we state it as fact. Where we are inferring behaviour that the schema only implies (for example, the precise lifecycle of an order, or the exact routing algorithm), we say so explicitly and justify the inference.
3. **Diagrams over prose where possible.** The eighteen TikZ figures are the spine of the document. They are designed to be readable on their own; the surrounding prose exists to explain, contextualise and cross-reference them, not to substitute for them.

1.3 Document Roadmap

Section 2 catalogues the artifact itself: the dump file, its metadata, the table prefix, the engine choices, the charset choices, the collation choices. Section 3 reconstructs the runtime stack (PHP/Apache or Nginx/MySQL/phpMyAdmin) that the dump’s header advertises. Section 4 identifies the architectural style as an OpenCart-derived Model-View-Controller (MVC) with several custom extensions, and introduces the four cross-cutting patterns (polymorphic object graph, EAV-style descriptions, multi-store scoping, multi-language scoping) that recur throughout the schema.

Section 5 is the longest. It walks through every functional domain — catalog, customers, orders, payments, banking, marketing, CMS, inventory, tax/geo, themes, tasks, users, stats, settings — and draws an Entity-Relationship diagram (ERD) for each. Section 6 extracts the patterns that span domains. Sections 7 through 9 reconstruct three runtime lifecycles (an HTTP request, a customer checkout, a task-scheduler tick). Sections 10 through 11 cover SEO/URL routing, the audit trail and the security/permission model. Section 12 discusses the data-integrity consequences of the MyISAM/no-foreign-keys choice. Section 13 closes with non-recommendatory

observations. Appendix A lists all 87 tables in a single long table for reference; Appendix B enumerates every polymorphic association in the schema.

2 Provenance and Artifact Description

2.1 The Repository

Table 1: Repository and artifact provenance.

Field	Value
Repository URL	https://github.com/yosietserga/necoyoad
Owner	yosietserga
Default branch	main
Total commits	1 (“Add files via upload”)
Files in repo	1 (necoyoad_db.sql, 76 467 bytes)
License	None declared
README	None
Source code	None — database dump only

2.2 The Dump File Header

The SQL file begins with a phpMyAdmin banner. The banner is more than ceremony: it pins down, with no ambiguity, the exact server configuration that produced the dump, and therefore the configuration against which the schema was, at minimum, known to compile. The relevant lines are reproduced verbatim below.

```
- phpMyAdmin SQL Dump
- version 5.2.0
- https://www.phpmyadmin.net/
-
- Host: localhost
- Generation Time: Feb 03, 2023 at 07:37 AM
- Server version: 5.7.17
- PHP Version: 8.1.2
```

2.3 What the Header Tells Us

Table 2: Inferred runtime stack from the dump header.

Layer	Version	Implication
phpMyAdmin	5.2.0	Administrative UI; release date 2022-11-22.
MySQL	5.7.17	Released 2016-12; EOL October 2023. <i>Legacy</i> .
PHP	8.1.2	Released 2021-12; supported at dump time.
Host	localhost	The dump was produced on the same host as the DB.
Timezone	+00:00	Dump taken in UTC; the running app likely uses a different local TZ.
SQL mode	NO_AUTO_VALUE_ON_ZERO	Inserts of 0 into AUTO_INCREMENT columns are preserved (not renumbered).
Charset	utf8mb4 (connection)	Full Unicode support at the connection layer.

Two facts deserve emphasis. First, MySQL 5.7 was already in the sunset of its supported life when the dump was produced (EOL was roughly eight months away). Second, the dump

explicitly sets `SET FOREIGN_KEY_CHECKS=0` at the top and restores it at the bottom. This is normal phpMyAdmin behaviour, but it also signals that the schema cannot rely on declared foreign keys for load-order safety — which, as Section 2.5 will show, is moot anyway because the tables are MyISAM and therefore cannot carry foreign-key constraints at all.

2.4 Database and Table Prefix

The database is named `necoyoad_db`. Every table is prefixed with the eight-character random string `nts8sd4fd_`; for example, `nts8sd4fd_product`, `nts8sd4fd_order`, `nts8sd4fd_user`. The prefix is a classic shared-hosting convention: it lets multiple applications share a single MySQL database without colliding on common table names like `user` or `setting`. The randomness of the prefix (`nts8sd4fd`) is mildly defensive — it makes the prefix harder to guess in a SQL-injection scenario, although this is a weak control by modern standards. Throughout this paper we will drop the prefix when discussing tables in prose (so “the `product` table”) but show it in full in code listings and in the appendix inventory.

2.5 Engine, Charset and Collation Choices

Every table in the dump is declared with `ENGINE=MyISAM`. This is a consequential choice with several knock-on effects:

- **No foreign keys.** MyISAM does not support `FOREIGN KEY` constraints. All referential integrity is the application’s responsibility. The schema’s many `_id` columns are therefore *soft* foreign keys — pointers that the database will not enforce.
- No transactions. MyISAM is non-transactional; a multi-table write (e.g. “insert an order, then insert its line items, then insert its totals, then decrement stock”) is not atomic. A crash midway leaves the database in a partially-applied state.
- Table-level locking. MyISAM locks the whole table for writes, which limits concurrency on write-heavy tables (notably `stat`, `search`, `user_activity`, `campaign_stat`, `campaign_link_stat`).
- Smaller disk footprint and faster full-table scans than InnoDB, which was the historical justification.

The charset story is inconsistent in a way that is itself diagnostic. The `DEFAULT CHARSET` of most tables is `latin1`, but individual columns are frequently overridden to `CHARACTER SET utf8 COLLATE utf8_bin`. This per-column override pattern is exactly what one sees when an existing `latin1` schema has been retrofitted for UTF-8 storage column by column, rather than migrated wholesale. It is a strong hint that the schema predates the application’s adoption of UTF-8.

The collation `utf8_bin` is binary and case-sensitive; this is unusual for user-facing string columns (which would normally use `utf8_general_ci` or `utf8_unicode_ci` for case-insensitive comparison). The choice has subtle consequences: an `email` lookup, for instance, would be case-sensitive at the database level, so the application must normalise email casing in application code before querying.

2.6 The 87-Table Inventory at a Glance

3 Inferred Runtime Stack

3.1 The Apache-or-Nginx / PHP / MySQL Triangle

The dump header pins MySQL 5.7.17 and PHP 8.1.2. The web server itself is not named, but two strong signals narrow the choice. First, phpMyAdmin 5.2.0 is installed; phpMyAdmin is

Table 3: Functional domains, table counts and indicative responsibilities.

#	Tables	Domain	Indicative responsibility
1	13	Catalog / Products	SKU catalogue, options, attributes, discounts, images, related items, types, manufacturers.
2	12	Orders & Payments	Order header, line items, totals, history, options, downloads, manual payments, coupons.
3	8	Customers & CRM	Customer accounts, groups, addresses, wallet balance, reviews, review likes.
4	9	Marketing / Campaigns	Newsletters, campaigns, contact lists, per-recipient tracking, link tracking.
5	4	Banking	Bank directory, account numbers, per-order manual transfer payments.
6	5	CMS / Content	Posts, categories, descriptions, banners, banner items.
7	7	Presentation	Menus, menu links, widgets, widget-to-landing-page, templates, themes, theme styles.
8	3	Task scheduler	Task definitions, queue, execution log.
9	6	Tax & Geography	Countries, zones, geo zones, tax classes, tax rates, zone-to-geo-zone.
10	4	Users & Permissions	Admin users, user groups, user activity log.
11	6	Core polymorphic spine	<code>object</code> , <code>object_to_category</code> , <code>object_to_store</code> , <code>description</code> , <code>property</code> , <code>status</code> .
12	5	Localisation	Languages, currencies, weight classes, length classes, URL aliases.
13	4	Audit & Analytics	<code>stat</code> , <code>search</code> , <code>notification</code> , <code>review_likes</code> .
14	2	Settings & Extensions	<code>setting</code> , <code>extension</code> .
15	1	Inventory	<code>warehouse_movement</code> .
	87	Total	

overwhelmingly deployed behind Apache with `mod_php`, or behind Nginx with PHP-FPM. Second, the schema’s `language` table contains a `directory` column and a `filename` column (each language points to a PHP language file on disk), which is the OpenCart convention and is framework-agnostic. We can therefore say with confidence that the runtime is some combination of $\{\text{Apache, Nginx}\} \times \{\text{mod_php, PHP-FPM}\} \times \text{MySQL 5.7}$, with PHP 8.1.2 as the interpreter. The exact variant does not materially affect any of the architectural conclusions below.

3.2 Why the Filesystem Still Matters

Although the dump is database-only, several schema columns tell us that the filesystem is a first-class data store. The `language` table has `directory` and `filename` columns; this is the OpenCart convention of loading `<directory>/<filename>.php` for translation strings. The `download` and `order_download` tables store a `filename` and a `mask` — the mask is what the user sees when downloading, the filename is the actual on-disk path. The `product_image`, `manufacturer_image`, `category_image`, `banner_item_image`, `bank_image`, `user_image`, `customer_photo` columns all hold filesystem paths. The `store.folder` column hints that multi-store is implemented by selecting a different filesystem folder per store. Any complete picture of the application must include this filesystem half; we will annotate it on each diagram where it is relevant.

3.3 Cron or Pseudo-Cron

The presence of a `task`, `task_queue`, and `task_exec` trio is unambiguous: there is a background job runner. Whether it is a real Unix `cron` entry that calls a PHP CLI script every minute, or a “pseudo-cron” triggered by the front controller on each page load (a common OpenCart-extension pattern), the schema alone cannot tell us. Section 9 discusses both options. The marketing-

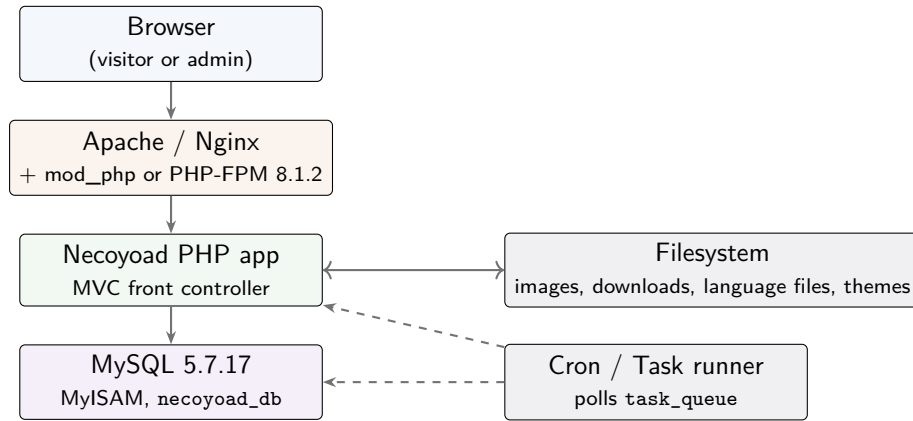


Figure 1: Inferred runtime stack. The browser hits the web server, which dispatches to a PHP process running the Necoyoad MVC application. The application reads and writes the `necoyoad_db` MySQL database and the filesystem (image uploads, downloadable products, per-language PHP translation files, theme assets). A separate cron process polls the `task_queue` table and re-enters the application to execute deferred jobs, primarily the marketing-campaign mailings.

campaign subsystem (Section 5.8) clearly relies on this scheduler: the `campaign_contact` table has a `task_queue_id` column that ties each recipient-to-campaign assignment to a queued job.

4 Architectural Style and Cross-Cutting Patterns

4.1 Style: OpenCart-derived MVC

The schema’s vocabulary is unmistakably OpenCart-derived. The following table maps OpenCart canonical table names to their Necoyoad counterparts and notes the divergences.

Table 4: OpenCart-to-Necoyoad table-name mapping (selected).

OpenCart (vanilla)	Necoyoad	Divergence
product	product	+ <code>owner_id</code> , <code>product_type</code> , <code>cost</code>
product_description	description	Generalised to polymorphic (<code>object_id</code> , <code>object_type</code>)
product_to_category	product_to_category + object_to_category	Both specific and polymorphic versions exist
category_description	description	Same polymorphic table
category_path	—	Not present; categories use <code>parent_id</code> only
customer	customer	+ <code>rif</code> , <code>company</code> , <code>photo</code> , <code>birthday</code> , <code>banned</code> , <code>activation_code</code>
order	order	+ <code>rif</code> , <code>coupon_id</code>
order_total	order_total	identical
url_alias	url_alias	+ <code>object_id</code> , <code>object_type</code> (polymorphic)
setting	setting	identical layout (<code>group</code> , <code>key</code> , <code>value</code>)
extension	extension	Heavily extended (<code>app</code> , <code>key</code> , <code>license</code> , <code>install</code> , <code>uninstall</code>)
coupon / coupon_history / coupon_product	identical	—

From this we infer the runtime architecture is a Model-View-Controller arrangement with OpenCart’s characteristic folder layout: a `catalog/` side (the customer-facing store), an `admin/` side (the back-office), a `system/` library folder, and a `image/`, `download/` and per-theme `catalog/view/theme/<theme>/` folder tree. The MVC request lifecycle is reconstructed in Section 7.

4.2 Layered View

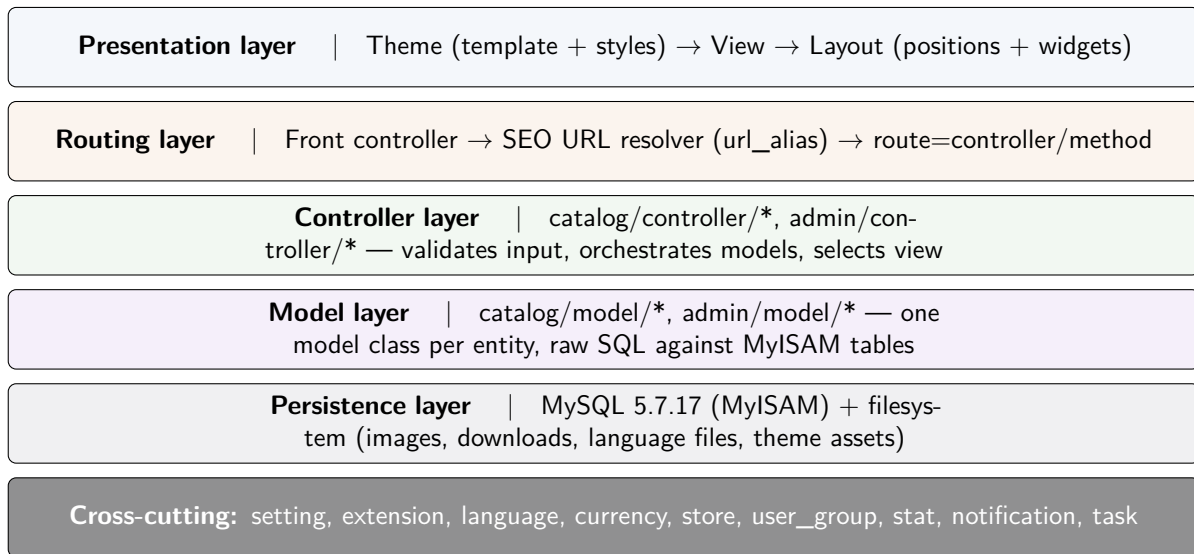


Figure 2: Layered view of the Necoyoad architecture. The presentation, routing, controller, model and persistence layers form the standard OpenCart-derived MVC pipeline. The bottom band lists cross-cutting concerns: settings drive configuration everywhere; extensions modify controller/model behaviour via the event/hook system; language and currency are loaded once per request; the stat table is written on every page impression; the notification table is read on every page load to surface admin/customer alerts; the task scheduler runs out of band.

4.3 Four Cross-Cutting Patterns

The schema exhibits four patterns that recur in domain after domain. We introduce them here because they are the conceptual key to the rest of the paper; their concrete realisations are enumerated in Section 6.

4.3.1 Pattern 1 — Polymorphic Object Graph

A handful of tables are written in a deliberately entity-agnostic way. The flagship is `object`, whose columns (`object_id`, `object_type`, `parent_id`, `parent_type`, `status_id`, `subtype`, `status`, `params`) are explicitly designed to hold *any* kind of content: a product, a post, a survey, an auction, etc. The `object_type` column is the discriminator. Around `object` are four companion tables that follow the same (`object_id`, `object_type`) pair as a composite soft foreign key: `object_to_category`, `object_to_store`, `description`, `property`, `url_alias`, `review`, `stat`, `notification`. In total there are at least eleven such polymorphic associations in the schema; they are enumerated in Appendix B.

4.3.2 Pattern 2 — EAV-style Descriptions

The `description` table is, in effect, an Entity-Attribute-Value table specialised for localised rich-text content. The “entity” is the polymorphic (`object_id`, `object_type`) pair; the “value” is a row containing `title`, `description` (rich HTML), SEO fields (`seo_title`, `meta_description`, `meta_keywords`) and a free-form `params` text. The “attribute” dimension is the `language_id`. This single table replaces what would, in vanilla OpenCart, be a dozen separate `<entity>_description` tables.

4.3.3 Pattern 3 — Multi-Store Scoping

The `store` table lists stores. Each store is identified by an integer `store_id` and has a `folder` (filesystem root for that store's assets) and an `owner_id`. The `customer`, `menu`, `notification`, `product_attribute`, `review`, `search`, `stat`, `task`, `template`, `theme`, `widget` tables all carry a `store_id` column, and the polymorphic junction `object_to_store` allows any content object to be visible in zero, one, many or all stores. The `setting` table is also keyed on `store_id`, so each store can have its own configuration overrides. This is a “multi-tenant within one database” model.

4.3.4 Pattern 4 — Multi-Language Scoping

The `language` table is the master list of supported languages. The `description`, `url_alias`, `product_option_description`, `product_option_value_description` and `product_tags` tables are all keyed on `language_id`. The `order` table records the `language_id` at order time so that invoices can be re-rendered in the customer's original language. The combination of multi-store and multi-language gives the system a $S \times L$ matrix of presentation surfaces.

4.4 Status Convention

A surprisingly important convention is the use of a `status` column almost everywhere, with remarkably varied semantics:

- `int(1)` (0 or 1) on most tables: simple on/off.
- `enum(' -1', '0', '1', '')` on `object`: a 3-state “deleted / disabled / enabled” lifecycle, with an empty-string state for backwards compatibility.
- `varchar(255) DEFAULT '1'` on `status` (the table): a free-form status code that other tables reference via `status_id`. This is the closest the schema comes to a normalised status table.
- `int(5)` on `order_history.order_status_id`: a numeric reference into a status lookup (which in this dump is not present as its own table, suggesting it lives in `setting` or in code).

The lack of a single, uniform status convention is itself a finding: different modules grew their own conventions over time.

5 Schema Walkthrough by Domain

This section is the heart of the document. For each functional domain we give: a short narrative, an Entity-Relationship Diagram (ERD) drawn in TikZ, and a table-by-table commentary on the columns that are not self-explanatory. ERDs use the following conventions: solid boxes are entity tables; dotted boxes are junction/lookup tables; arrows are soft foreign keys (MyISAM has no real ones); the `(object_id, object_type)` polymorphic pair is drawn with a double-headed arrow.

5.1 Domain 1 — The Polymorphic Object Spine

Before any specific domain, we must lay out the polymorphic spine, because every other domain hangs off it. The spine consists of six core tables (`object`, `description`, `property`, `status`, `object_to_category`, `object_to_store`) and four satellite tables that are polymorphic by convention (`url_alias`, `review`, `notification`, `stat`).

The `object` table is the most interesting table in the entire schema. Its columns explicitly document, via SQL comments, the intent:

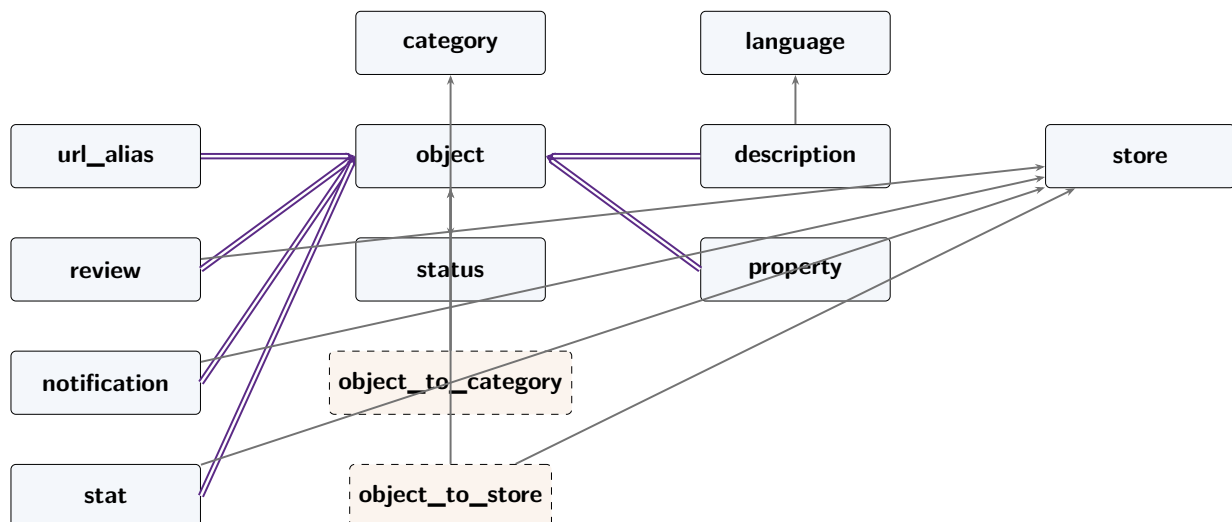


Figure 3: The polymorphic object spine. Six core tables and four satellite tables all reference the object table via the polymorphic pair (object_id, object_type), shown as a double-headed purple arrow. The category and store tables are referenced both directly and indirectly. The status table is the normalised lookup for state codes.

```

object_id INT - primary key
object_type VARCHAR(100) - 'product', 'post', 'survey', ...
parent_id INT - tree support (categories, subcategories)
parent_type VARCHAR(100) - 'product_category', 'post_category', ...
status_id INT - FK to status table
subtype VARCHAR(100) - 'digital', 'normal', 'auction', ...
status ENUM('-1','0','1','') - -1:deleted, 0:disabled, 1:enabled
params TEXT - 'anything, arrays, classes, objects, strings'
sort_order INT
date_added TIMESTAMP DEFAULT CURRENT_TIMESTAMP
date_modified DATETIME

```

The comment on `params` — “anything, arrays, classes, objects, strings” — is the most candid admission in the entire schema: this column is a serialised blob of arbitrary PHP data, almost certainly `serialize()/unserialize()` or `json_encode()/json_decode()`. The same pattern recurs on `banner.params`, `customer_group.params`, `extension.settings`, `widget.settings`, `menu.route`, `product_attribute.params` (no, wait — `product_attribute` has no params; but `property` has value which is explicitly `TEXT` and used for “anything”). At least seven tables carry a `params` or `settings` `TEXT` column whose payload is a PHP serialised value. This is a deliberate escape hatch from the relational model and is the second cross-cutting pattern (after polymorphic associations).

5.2 Domain 2 — Catalog and Products

The catalog domain is the largest functional area, with thirteen tables. It is also the most OpenCart-flavoured: the table names (`product`, `product_to_category`, `product_option`, `product_option_value`, `product_discount`, `product_special`, `product_image`, `product_related`, `product_tags`, `product_to_download`, `manufacturer`) are essentially identical to OpenCart 1.5/2.x, with the addition of `product_type`, `product_attribute`, `product_attribute_group`, and `product_to_zone`.

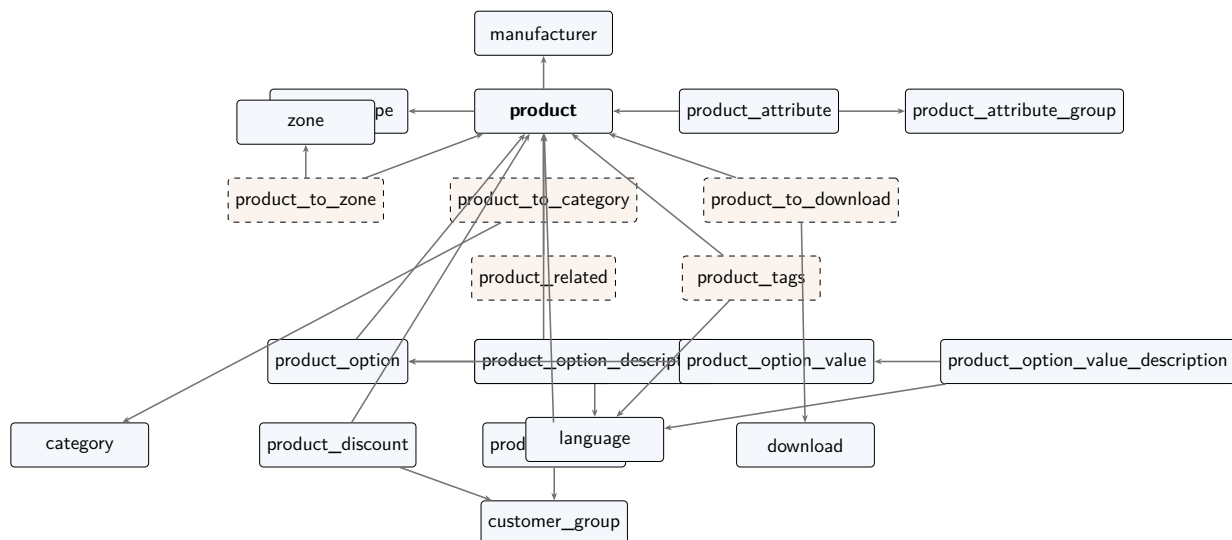


Figure 4: Catalog domain ERD. The product table is the hub. Thirteen satellite tables cover taxonomies, localisation, inventory-by-zone, media, merchandising and extensibility. See the body text for the full per-table breakdown.

5.2.1 Notable Columns in product

- **owner_id** — the customer or admin who owns the product. This is non-OpenCart and suggests a vendor/marketplace model where customers can be sellers.
- **product_type** VARCHAR(255) — a free-form type discriminator (digital, normal, auction, ...). It is also backed by the **product_type** lookup table; the relationship between the two is not enforced.
- **sku**, **model** (unique), **location** — standard inventory identifiers.
- **quantity**, **subtract**, **minimum**, **stock_status_id** — standard OpenCart stock controls.
- **price** (gross) and **cost** — both stored, so the system can compute margin in reports.
- **weight**, **length**, **width**, **height** plus their class IDs — physical dimensions for shipping.
- **date_available** — the date from which the product is listed. Combined with **status**, this gives a time-windowed visibility model.
- **viewed** — a denormalised view counter, incremented per product page load.
- **sort_order** — manual ordering.

5.2.2 The Three Pricing Tiers

The catalog has three distinct pricing mechanisms that layer on top of the base **product.price**:

1. **product_discount** — quantity breaks scoped to a **customer_group_id** and a date window (**date_start**, **date_end**). Standard OpenCart.
2. **product_special** — a promotional price, also scoped to a **customer_group_id** and a date window. Standard OpenCart.
3. **coupon** (see Section 5.4) — a checkout-time discount applied via a code.

The resolution order at display time is, by OpenCart convention: **special** (if active) → **discount** (if quantity threshold met) → base **price**. The application is responsible for this resolution; the database stores all three independently.

5.2.3 Product Attributes vs. Product Options

The schema distinguishes two easily-confused concepts:

- **Product attributes** (`product_attribute` + `product_attribute_group`) are *display-only* specifications: e.g. “Screen size: 5.5in”, “Battery: 3000mAh”. They are defined per-store (there is a `store_id` column), grouped, typed (`type`, `pattern`, `default`, `required`) and are not directly orderable. The `group`, `label`, `name` columns imply a form-builder UI.
- **Product options** (`product_option` + `product_option_description` + `product_option_value` + `product_option_value_description`) are *orderable* variants: e.g. “Colour: Red / Blue”, “Size: S / M / L”. Each value has its own `quantity`, `subtract` flag, `price` delta and `prefix` (+ or -). They are language-translated via the two `_description` tables.

5.2.4 Tags and Taxonomy

`product_tags` (note the plural — OpenCart convention) is a separate, language-keyed tag table. The primary key is the composite (`product_id`, `tag`, `language_id`), which prevents duplicate tags per language but allows the same tag in different languages. The `product_to_category` junction is the primary taxonomy; `product_to_zone` restricts a product’s availability to certain zones (likely for shipping or regulatory reasons); and `product_related` is a self-referential many-to-many “related products” table.

5.3 Domain 3 — Customers, Addresses and Reviews

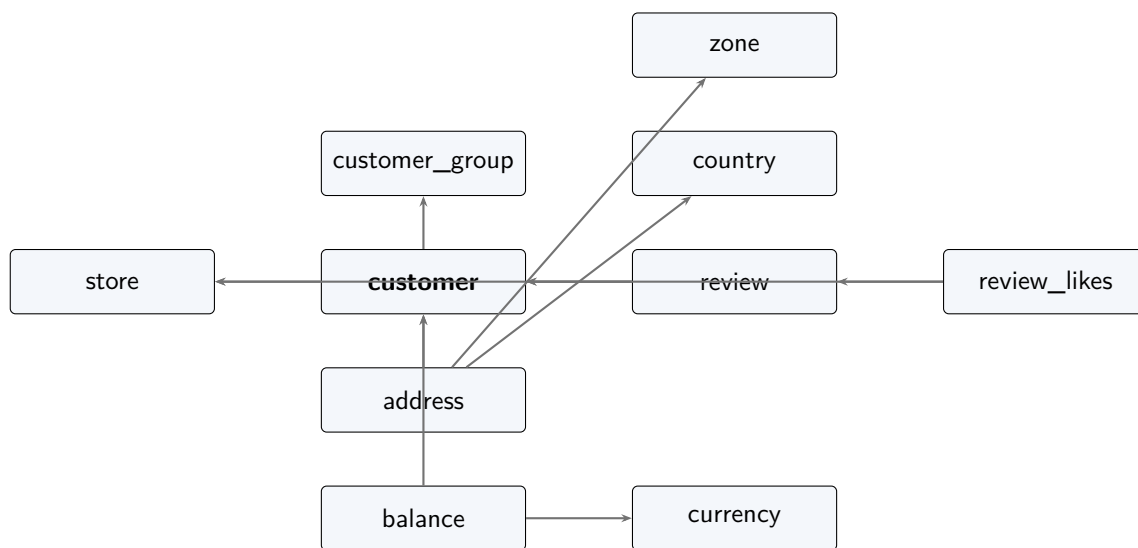


Figure 5: Customer domain ERD. A customer belongs to one `customer_group` (drives pricing and permissions), is registered against a `store`, and has one or more `addresses`. The `balance` table is a per-currency wallet ledger; `review` and `review_likes` form a small social layer.

5.3.1 Notable Columns in `customer`

The `customer` table is significantly extended beyond vanilla OpenCart. The Venezuelan context is visible in several places:

- `rif` `VARCHAR(12)` — the Venezuelan *Registro de Información Fiscal* tax identifier, used for both individuals and companies. This is a strong signal that the application was built for the Venezuelan market.

- **company** VARCHAR(90) — the customer’s company name (relevant because the RIF can be corporate).
- **activation_code** VARCHAR(255) — the account-activation token sent by email at signup.
- **photo** VARCHAR(255) — a filesystem path to the customer’s avatar.
- **birthday, congrats** — birthday and a flag for whether the customer has been sent birthday greetings (likely via the campaign system).
- **referenced_by** — a customer-referral field.
- **cart** TEXT — a serialised PHP array of the customer’s shopping cart, persisted between sessions. This is the OpenCart convention.
- **banned, approved, complete, visits, ip** — moderation and audit fields.
- **sex** CHAR(1) — gender.
- **password** VARCHAR(100) — the password hash (the length suggests bcrypt or Argon2; the legacy OpenCart md5+salt would fit in 40 characters).

5.3.2 The Wallet: balance

The **balance** table is a per-customer, per-currency ledger. Each row records: a **type** (3-char code, likely **cr** for credit and **db** for debit), an **amount**, a **description**, three running totals (**amount_available**, **amount_blocked**, **amount_total**) and a currency snapshot (**currency_code**, **currency_value**, **currency_title**). The presence of a “blocked” amount suggests the wallet can have funds reserved (e.g. for a pending order) and released (when the order completes) or voided (when the order is cancelled). This is the mechanism that ties the wallet to the order-payment flow (Section 5.5).

5.3.3 Reviews and Likes

The **review** table is polymorphic: it can attach to any **object_id/object_type** pair (a product, a post, etc.). It has a **parent_id** for threaded replies, an **author** string (so non-customers can also review), a **rating** (1–5), and the standard status / date fields. The **review_likes** table records per-customer like/dislike votes on reviews, with a unique key on (**review_id**, **customer_id**) to prevent double-voting.

5.4 Domain 4 — Orders, Coupons and Checkout

5.4.1 The Snapshot Pattern in order

The **order** table is a textbook example of the *snapshot* pattern: rather than normalising the customer’s name, address, phone, email, shipping address and payment address into references to the **customer** and **address** tables, the order copies all of these fields verbatim into its own columns (**firstname**, **lastname**, **telephone**, **rif**, **email**, **shipping_firstname**, **shipping_lastname**, ..., **payment_firstname**, **payment_lastname**, ...). This is standard practice in commerce systems: an order is a legal artifact and must remain readable even if the customer later changes their name or deletes their account. The trade-off is a wide, denormalised table; the **order** table has 43 columns.

5.4.2 Order Status and History

The **order** table has a single **order_status_id** column (current status). The full status history is kept in **order_history**, where each row records an **order_status_id** change, a **notify** flag (whether the customer was emailed about this transition), a **comment** and a timestamp. This is a standard OpenCart state-log pattern.

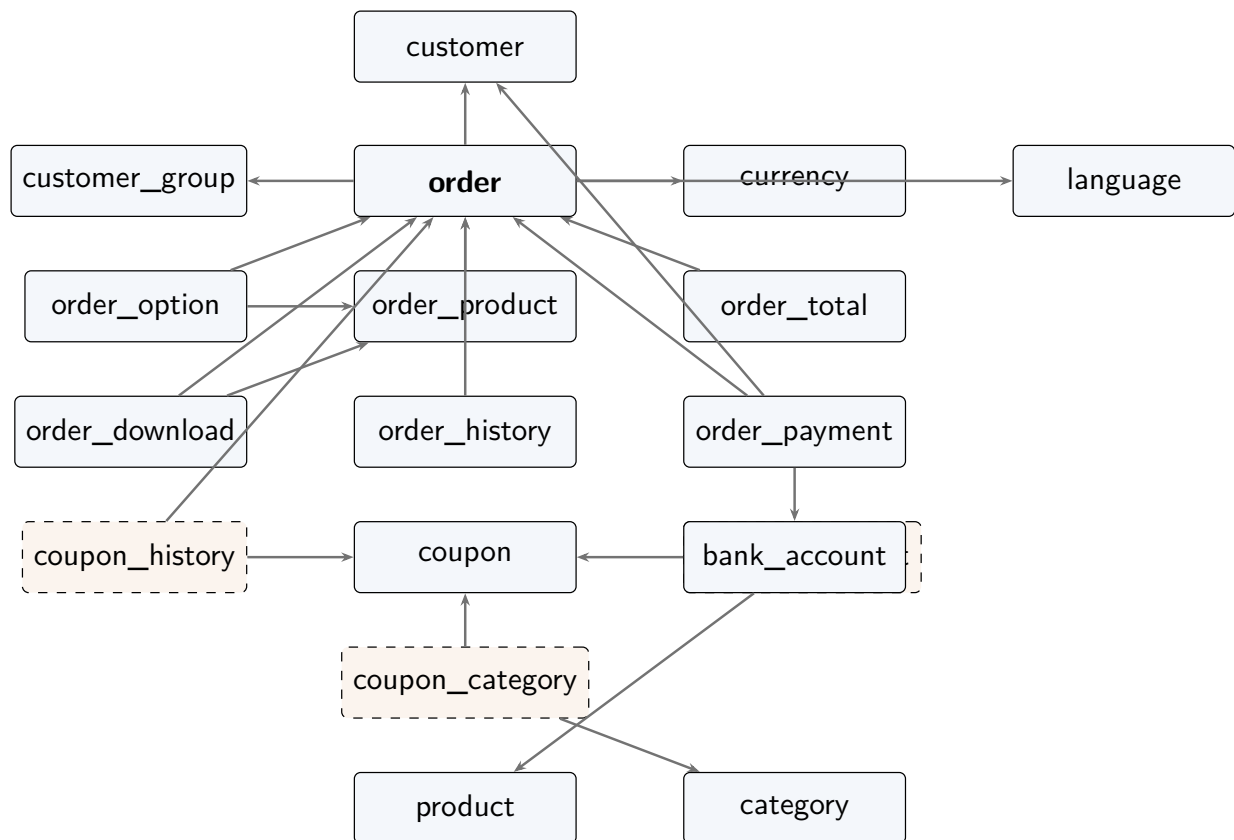


Figure 6: Order and coupon ERD. The order table is the hub. It snapshots the customer, address, shipping and payment details at order time. Each order_product line carries its options (via order_option) and any associated downloads (via order_download). The order_total table holds the ordered list of total-line computations. The order_payment table records manual bank-transfer payments. Coupons are scoped to products and categories, and have a usage history.

5.4.3 order_total — The Extension Point

The `order_total` table is the database side of one of OpenCart’s most important extension points. Each row is a single line in the cart total: `title` (e.g. “Sub-Total”), `text` (the formatted, localised amount, e.g. “USD 120.00”), `value` (the numeric amount), `sort_order`. The application computes these lines by iterating over a set of “total extension” classes (sub-total, shipping, tax, coupon, voucher, reward points, ...) installed via the `extension` table. Each extension contributes one or more rows to `order_total` when the order is finalised. The `order.total` column is the sum of all `order_total.value` rows for that order.

5.4.4 order_payment — Manual Bank Transfers

The `order_payment` table records manual payment notifications: the customer reports that they have made a bank transfer (transaction number, date, originating bank, amount, comment), and an admin later marks the row as confirmed by setting its `order_payment_status_id`. Each row links to a `bank_account` (the merchant’s account that received the transfer). This is the typical flow in Venezuelan e-commerce, where credit-card payment gateways were unreliable for years and bank transfers became the default.

5.4.5 Coupons

The `coupon` table is the OpenCart standard: a code, a discount `type` (P percentage or F fixed), a discount amount, a total minimum cart total, a date window, `uses_total` (max global uses), `uses_customer` (max uses per customer, stored as `VARCHAR(11)` — possibly “” or “N”), a `logged` flag (must the customer be logged in?) and a `shipping` flag (does the coupon free shipping?). The `coupon_product` and `coupon_category` junctions scope the coupon to specific products or categories. The `coupon_history` table logs every use.

5.5 Domain 5 — Banking and Wallet

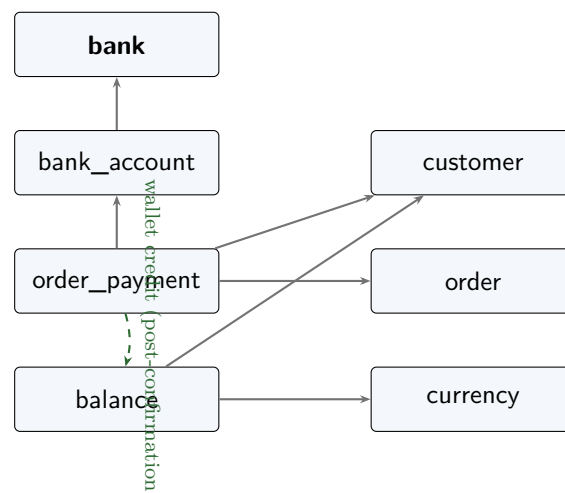


Figure 7: Banking and wallet domain. The merchant maintains a directory of banks (`bank`) and their accounts (`bank_account`). The customer reports a transfer via `order_payment`; after admin confirmation, the wallet `balance` for that customer is credited (dashed green arrow — inferred post-confirmation side-effect, not enforced by the database).

The banking domain is small but operationally important. The `bank` table holds the master list of banks (name, logo image, dates). The `bank_account` table records each merchant account: the owning `bank_id`, the account `number` (unique), the `accountholder` name, the `type` (savings,

checking, ...), the contact **email**, the **rif** (the merchant’s tax ID for that account), and a **status**. The **order_payment** table, described in Section 5.4, references **bank_account**.

The dashed arrow in Figure 7 is an inferred side-effect: when an admin confirms an **order_payment** (by advancing its **order_payment_status_id**), the application is expected to also write a corresponding credit row to the customer’s **balance** ledger. The database does not enforce this; it is an application-level invariant.

5.6 Domain 6 — CMS, Posts, Banners, Menus

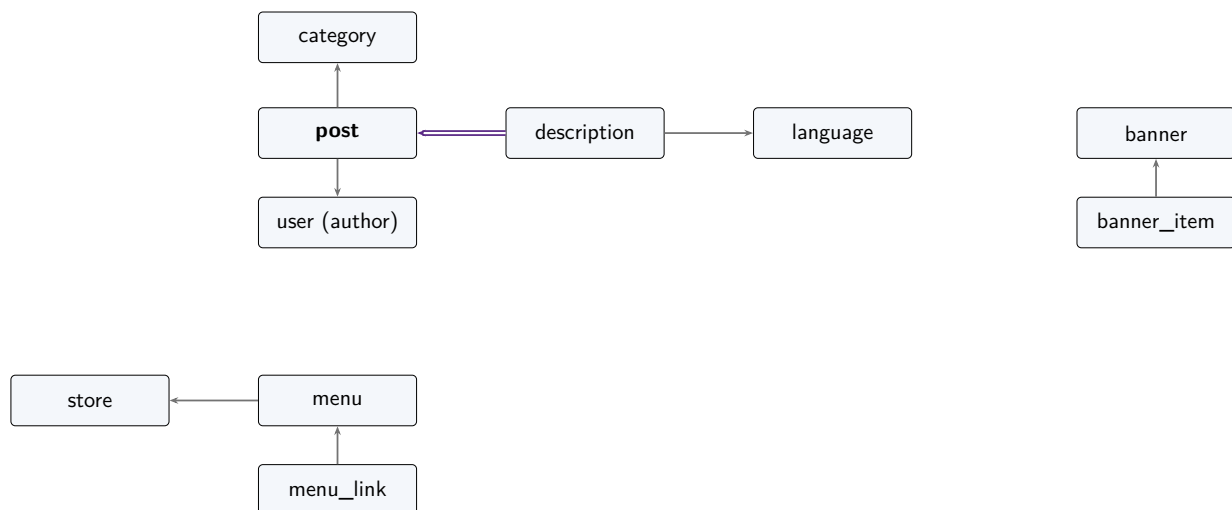


Figure 8: CMS domain. A **post** is a content node (blog article, page, custom post type) authored by an admin **user** and classified by a polymorphic **category**. Its localised title/body/SEO fields live in the **description** table (polymorphic, keyed on **object_id** = **post_id**, **object_type** = 'post'), one row per language. Banners and menu structures are independent subsystems.

5.6.1 Posts

The **post** table is a small but flexible CMS engine. Each post has: a **parent_id** (for hierarchical post types, e.g. a page tree), an **author_id** (admin user), a **post_type** discriminator (e.g. **post**, **page**, **article**), an image, a publishing window (**date_publish_start**, **date_publish_end**, **publish** flag), an **allow_reviews** flag, and a **template** column that selects which view template renders the post. The combination of **post_type** + **template** gives the system a custom-post-type mechanism reminiscent of WordPress.

5.6.2 Banners

The **banner** table is interesting because of its **jquery_plugin** and **params** columns. Each banner can specify a jQuery slideshow plugin (e.g. “nivo- slider”, “bxslider”) and a serialised parameter set. This is a deliberate extension point: the front-end renders the banner by loading the named jQuery plugin and feeding it the banner items and the params. Each **banner_item** is a single slide (image + link + sort order + status).

5.6.3 Menus

The **menu** table stores named menus scoped to a store (**store_id**), with a **position** (e.g. “header”, “footer”, “sidebar”), a **route** (the page the menu links to by default), a **default** flag and a **status**. The **menu_link** table stores the individual menu items, with a **parent_id** for tree-structured menus, a **link**, a **tag** (the visible label) and a **sort_order**.

5.7 Domain 7 — Themes, Templates and Widgets

The presentation layer is a four-table stack that gives the system a complete visual-customisation surface: **template** (the design package), **theme** (an instance of a template, scoped to a store and a user), **theme_style** (CSS-rule-level overrides for a theme), and **widget** (positioned, configurable content blocks). A fifth table, **widget_landing_page**, binds a widget to a specific landing page and object type.

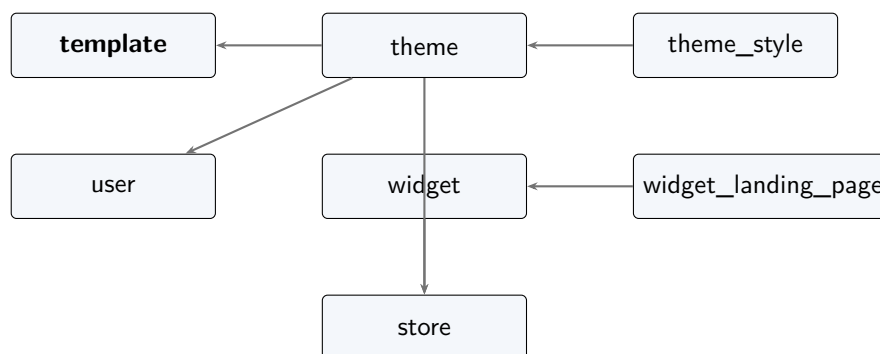


Figure 9: Presentation stack. A **template** is a design package (with versioning and a “for_nt_version” compatibility field). A **theme** is an instantiation of a template for a particular store and editor (**user_id**), with a default flag and a publishing window. A **theme_style** row is a single CSS rule override (selector, property, value) for a theme. A **widget** is a positioned, configurable content block; the **widget_landing_page** table restricts a widget to a particular landing page and object type.

5.7.1 The Template Versioning Column

The **template** table has a **for_nt_version** column. “nt” almost certainly stands for “Necoyoad Template” or “Necoyoad Theme”. The presence of this column tells us that the platform has its own version numbering, separate from OpenCart’s, and that templates declare which platform version they target. This is consistent with the system being a fork or derivative rather than a vanilla OpenCart install.

5.7.2 Widgets

The **widget** table is one of the most application- rich tables in the schema. Each widget has: **code** (a unique identifier), **name**, **extension** (which extension provides this widget’s controller), **position** (where on the page it appears), **app** (which app: “catalog”, “admin”), **order** (sorting), **settings** (a serialised PHP blob of widget configuration), and **status**. The combination of **extension** and **settings** makes widgets the primary extension surface for the front-end: an admin can drop a widget into a position, configure it, and the front controller dispatches to the widget’s controller to render it.

5.8 Domain 8 — Marketing and Campaigns

The marketing domain is the most sophisticated subsystem in the schema after the catalog. It implements a complete email-marketing pipeline: newsletters, campaigns, contact lists, per-recipient tracking, link click tracking, and a queue-based sending engine that runs on the task scheduler.

5.8.1 The Campaign Lifecycle

A campaign has a **date_start**, a **date_end**, a **repeat** field (likely “once”, “daily”, “weekly”, “monthly”), and several tracking flags: **trace_email** (track opens), **trace_click** (track clicks), **embed_image** (embed images vs. link them). The inferred lifecycle is:

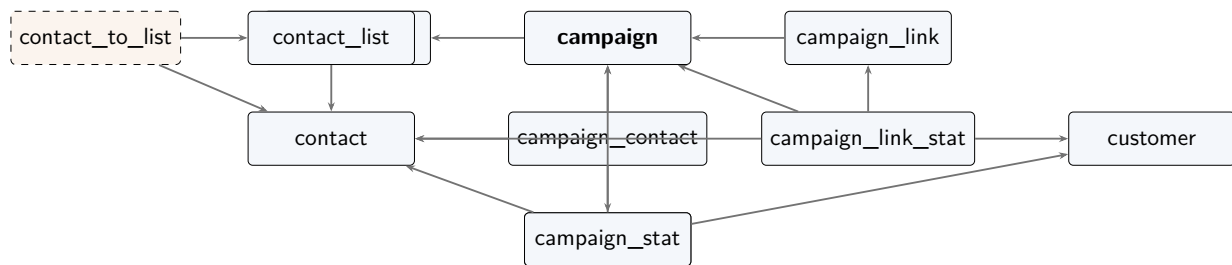


Figure 10: Marketing/CRM domain. A **newsletter** is a reusable email body (text + HTML). A **campaign** is a sending of a newsletter to a set of contacts. Each **campaign_contact** row is one recipient of one campaign, linked to a **task_queue** entry that drives the actual send. **campaign_link** stores the rewritten, trackable links inside the email. **campaign_stat** records email opens (per recipient, with full HTTP context). **campaign_link_stat** records link clicks (per recipient, per link). Contacts are organised into **contact_lists** via the **contact_to_list** junction.

1. Admin composes a **newsletter** (text + HTML body).
2. Admin creates a **campaign** that uses the newsletter and targets one or more **contact_lists**.
3. At campaign creation time, the application expands the target lists into individual **campaign_contact** rows — one per **contact**. For each, it creates a **task_queue** entry that will perform the actual send at the campaign's **date_start** (or per the **repeat** schedule).
4. For each **campaign_contact**, the application also rewrites the newsletter's links into trackable **campaign_link** URLs.
5. When the task fires, the application sends the email. When the recipient opens it, a tracking pixel hits a URL that creates a **campaign_stat** row. When the recipient clicks a link, the redirect handler creates a **campaign_link_stat** row and redirects to the real URL.

5.8.2 The Tracking Schema

Both **campaign_stat** and **campaign_link_stat** record a remarkably rich HTTP context: **store_url**, **server** (serialised **\$_SERVER**), **session** (serialised **\$_SESSION**), **request** (serialised **\$_REQUEST**), **ref** (referrer), **browser**, **browser_version**, **os**, **ip**. This is clearly a do-it-yourself analytics layer, capturing everything the PHP process knows about the request at tracking time. The same pattern recurs in the **stat**, **search** and **user_activity** tables — see Section 5.14.

5.9 Domain 9 — Inventory and Warehouse

The inventory domain is, surprisingly, a single table: **warehouse_movement**. There is no **warehouse** or **shelf** master table; their IDs are referenced (**warehouse_id**, **shelf_id**) but not enforced. This is either a deliberate simplification (warehouses and shelves are configured in code or in **setting**) or an indication that the inventory subsystem was a work in progress at dump time.

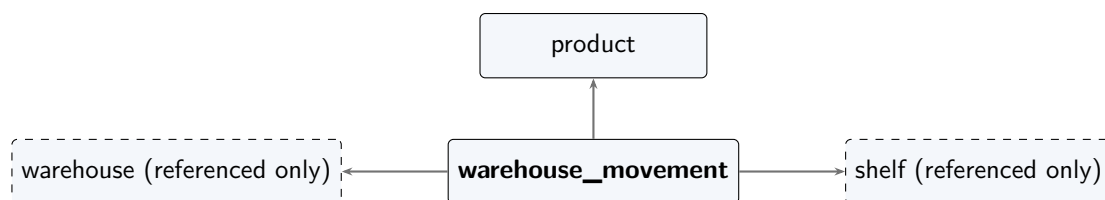


Figure 11: Inventory domain. A single table **warehouse_movement** records every movement of stock (in, out, transfer). **warehouse** and **shelf** are referenced by ID but have no master table in this dump.

The `warehouse_movement` table records: `product_id`, `warehouse_id`, `shelf_id`, `batch_number`, `unit`, `weight`, `qty`, `movement` (the type of movement), `rotation` (likely FIFO/LIFO/FEFO), `barcode_number`, `barcode_type`, `date_leave`, `date_expiration`, plus the standard `date_added/date_modified`. This is a complete batch-and-expiry inventory movement log, suitable for food, pharmaceuticals or any regulated goods where traceability matters.

5.10 Domain 10 — Tax and Geography

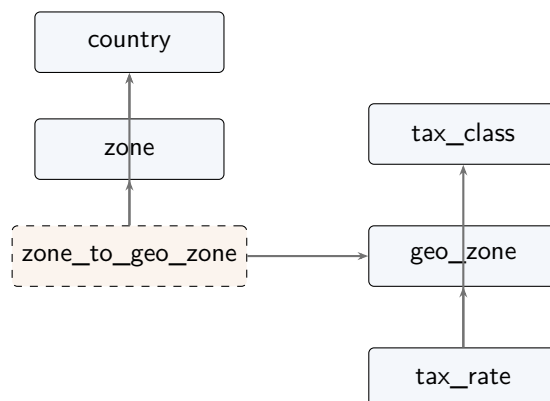


Figure 12: Tax and geography ERD. Countries contain zones (states/ provinces). Zones (or whole countries) are grouped into `geo_zones` via `zone_to_geo_zone`. Each `tax_rate` applies within a `geo_zone` and belongs to a `tax_class`. The catalog (`product.tax_class_id`) references `tax_class`. This is the standard OpenCart tax model.

The tax/geography domain is exactly OpenCart’s. Countries have ISO codes (`iso_code_2`, `iso_code_3`) and an `address_format` template. Zones belong to countries. A many-to-many `zone_to_geo_zone` table groups zones into “geo zones” for tax-rule purposes (e.g. “Andean region”, “Caribbean region”). `tax_class` groups related tax rates (e.g. “Taxable Goods”). `tax_rate` defines a rate, scoped to a geo zone, with a priority. The application computes the applicable tax for a cart by looking up the customer’s shipping zone, finding the matching geo zones, finding the tax rates for the products’ tax classes within those geo zones, and applying them in priority order.

5.11 Domain 11 — Users, Groups and Audit

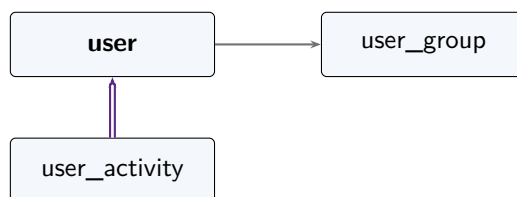


Figure 13: User/permission domain. Admin users belong to a `user_group` whose `permission` column is a serialised PHP structure of allowed `route=...` patterns. The `user_activity` table is a polymorphic audit log (recorded against any `object_id/object_type`).

The `user` table is the admin-staff account table (distinct from `customer`, which is the store-front account). Each user has a `user_group_id` whose `permission` column is a TEXT blob. The content of this blob, by OpenCart convention, is a serialised PHP associative array with two keys, `access` and `modify`, each containing a list of `catalog/*` or `admin/*` route patterns the group is allowed to access or modify. The front controller checks the current user’s group `permission` against the requested route before dispatching.

The `user_activity` table is the admin audit log. Each row records: `user_id`, the polymorphic `object_id/object_type` pair, an `event` (e.g. login, logout, add, edit, delete), an `action`, a free-text `description`, the `ip`, `browser`, `browser_version`, `os`, `session` (serialised), and a timestamp. This is the same HTTP-context snapshot pattern as the marketing and stats tables.

5.12 Domain 12 — Tasks and Scheduler

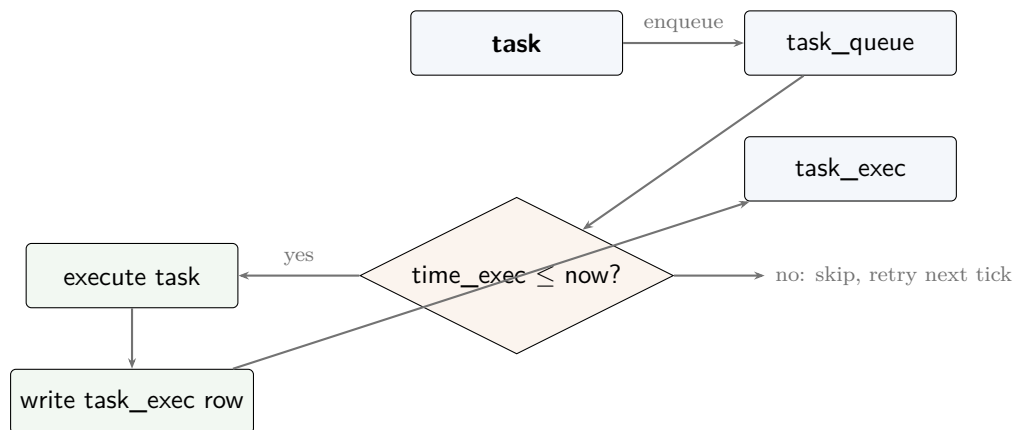


Figure 14: Task scheduler. The `task` table holds the template (what to run, on what cadence). The `task_queue` table holds the materialised, parameterised instances waiting to run. The `task_exec` table is the audit log of past executions. The runner (cron or pseudo-cron) polls `task_queue` for rows whose `time_exec` has passed, executes them, and writes a `task_exec` row.

The `task` table records: `store_id`, `object_id/object_type` (the entity the task operates on), `task` (a name identifying the task handler), `type`, `params` (serialised), the schedule (`time_interval`, `time_exec`, `time_last_exec`, `run_once`), `status`, `sort_order`, and execution-window columns (`date_start_exec`, `date_end_exec`). Note the typo `data_added` (instead of `date_added`) — a small but real artifact of the system’s hand-built evolution.

The `task_queue` table is the per-instance queue: `task_id`, `params` (per-run parameters, can override the task template’s), `time_exec` (when to run), `status`, `sort_order`. The `task_exec` table records: `task_id`, `type`, `status`, `date_added`. The runner picks up due rows, runs them, writes a `task_exec` row, and updates the `task_queue` row’s status.

The marketing-campaign subsystem uses this scheduler directly: each `campaign_contact` row references a `task_queue_id`, so the campaign send is literally a queue of tasks, one per recipient.

5.13 Domain 13 — Localisation

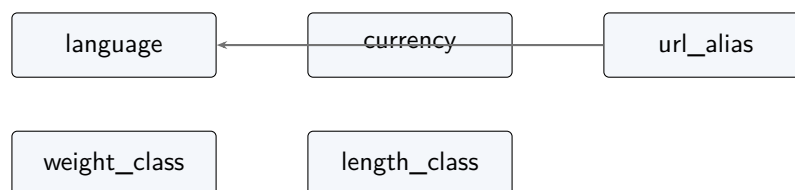


Figure 15: Localisation domain. The four unit-class tables (`language`, `currency`, `weight_class`, `length_class`) are reference lookups. The `url_alias` table is the SEO layer: it maps human-readable keywords to internal (`object_id`, `object_type`, `language_id`) triples.

5.13.1 Language

The `language` table is small but expressive: `name`, `code` (e.g. “en-gb”, “es-ve”), `locale` (a full locale string), `image` (a flag icon path), `directory` and `filename` (the PHP language file to load). This last pair is the OpenCart convention: translations live in `catalog/language/<directory>/<filename>.php` and are loaded into a `$_` array at request startup.

5.13.2 Currency

The `currency` table records each currency’s `code`, `symbol_left`, `symbol_right`, `decimal_place`, and `value` (the exchange rate relative to the base currency). The `date_modified` column on `currency` is the last time the rate was updated. There is no built-in auto-update mechanism; the application either polls an external rate service via a task, or expects an admin to update rates manually.

5.13.3 url_alias — The SEO Layer

The `url_alias` table is the system’s SEO-URL layer. Each row maps a `keyword` (the slug, e.g. “my-first-product”) to an internal `query` string (e.g. `product_id=42`), scoped by `object_id`, `object_type` and `language_id`. Two unique keys enforce correctness: `keyword + language_id` must be unique (so the same slug cannot point to two objects in the same language), and `object_id + language_id + object_type` must be unique (so each object has at most one slug per language). This is a polymorphic, language-aware short-URL system — see Section 10 for the routing algorithm.

5.14 Domain 14 — Notifications, Stats and Search

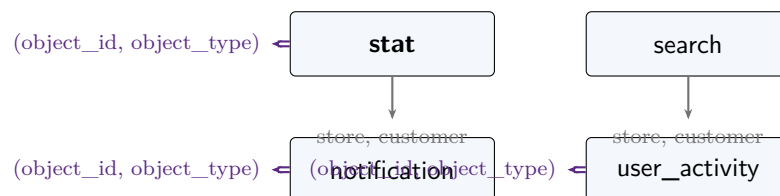


Figure 16: Audit and analytics domain. Four tables record per-request or per-action events: `stat` (page views), `search` (search queries), `notification` (admin/customer notifications), `user_activity` (admin audit log). All four carry a polymorphic (`object_id`, `object_type`) reference, and three (`stat`, `search`, `user_activity`) carry a full HTTP-context snapshot.

The audit/analytics domain is implemented as four independent event-log tables, all following the same general shape:

- **stat** — a row per page view, recording the `customer_id`, the polymorphic object being viewed, the `store_id/store_url`, the full HTTP context (`server`, `session`, `request`, `ref`), the user-agent details (`browser`, `browser_version`, `os`), the `ip`, and a timestamp. The `email` column hints that the application tries to attribute anonymous views to an email address when possible.
- **search** — a row per search query, recording the `urlQuery` (the search string and filters), the customer, the store, and the same HTTP context.
- **notification** — a per-user notification, polymorphic on the object that triggered it. Has a `message` column (described in the SQL comment as “Mensaje que sera traducido” — a message that will be translated, i.e. a translation key with parameters), a `url` for the user to click through to, and a `type`.

- **user_activity** — the admin audit log, described in Section 5.11.

The four tables are clearly part of the same conceptual family (per-event analytics with full HTTP context) but use slightly different column sets, suggesting they were added at different times by different code paths.

5.15 Domain 15 — Settings and Extensions



Figure 17: Configuration domain. The **setting** table is a flat key-value store, scoped by **store_id** and **group**. The **extension** table is a registry of installed extensions (payment, shipping, total, payment-report, etc.) with licensing metadata.

5.15.1 The Flat Settings Store

The **setting** table is OpenCart’s classic flat configuration store: each row is a (**store_id**, **group**, **key**, **value**) tuple. A row with **store_id** = 0 is the default; rows with **store_id** = **N** > 0 override the default for that store. The **group** typically names the extension (e.g. “config”, “shipping_flat”, **payment_bank_transfer**); the **key** is the parameter name (e.g. “geocode”, “telephone”, “flat_cost”); the **value** is a string. The application loads all settings for the current store at request startup and caches them in a registry.

5.15.2 Extensions

The **extension** table is the registry of installed extensions. It is significantly richer than vanilla OpenCart’s. Columns: **type** (the extension category: **payment**, **shipping**, **total**, **module**, **feed**, **analytics**, **report**, ...), **app** (which app: **catalog** or **admin**), **key** (the extension’s unique code), **license** (a license key), **install** and **uninstall** (URLs or controller routes for installation / uninstallation hooks), **url_developer** (the developer’s website), **settings** (serialised extension-specific settings), **version**, **status**, **last_update**, **date_added**. The presence of **license**, **install**, **uninstall** and **url_developer** strongly suggests a commercial extension marketplace: extensions can be bought, licensed, installed and uninstalled from the admin UI.

6 Cross-Cutting Patterns, Formalised

Section 4 introduced the four cross-cutting patterns in prose. Here we formalise them with a single combined diagram and an inventory of every table that participates in each.

6.1 Pattern Map

6.2 Polymorphic Association Inventory

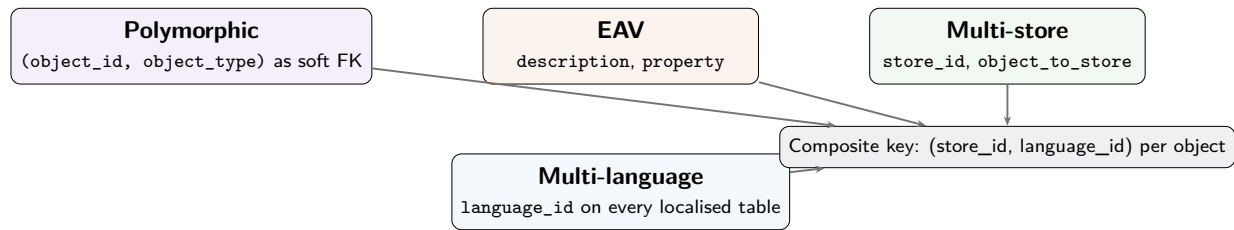


Figure 18: The four cross-cutting patterns compose. A single piece of content — say, a product’s Spanish description shown on Store 2 — is located by the composite key (`store_id=2`, `language_id=2`, `object_id=42`, `object_type='product'`).

Table 5: Every polymorphic (`object_id`, `object_type`) association in the schema.

Table	Polymorphic columns	What it points to
object	parent_id, parent_type	any other object (tree support)
object_to_category	object_id, object_type	any object to a category
object_to_store	object_id, object_type	any object to a store
description	object_id, object_type	any object’s localised text
property	object_id, object_type	any object’s free-form key-value
url_alias	object_id, object_type	any object’s SEO slug
review	object_id, object_type	any object’s review
review_likes	object_id, object_type	the review’s parent object
stat	object_id, object_type	the object being viewed
notification	object_id, object_type	the object that triggered the notification
user_activity	object_id, object_type	the object the admin acted on
task	object_id, object_type	the object the task operates on
widget_landing_page	object_type	the type of landing page
category	object_type	the kind of object the category groups
campaign_link_stat	(implicit via contact + campaign)	marketing context
campaign_stat	(implicit via contact + campaign)	marketing context

6.3 The “params” / “settings” Escape Hatch

At least seven tables carry a TEXT column whose payload is a serialised PHP value: `object.params`, `banner.params`, `customer_group.params`, `extension.settings`, `widget.settings`, `setting.value` (sometimes), `task.params`, `task_queue.params`, `product_attribute.params` (not present, but `property.value` plays the same role). This is a deliberate, schema-level escape hatch: anything that does not fit the relational model is stuffed into a serialised blob and decoded in PHP. The cost is opacity: from the SQL side, one cannot query, index or join on the contents of these blobs. They are a frequent source of bugs when the serialised format changes between PHP versions (e.g. PHP 7’s `serialize()` of Closure objects differs from PHP 8’s).

6.4 Composite Conventions

Three column-pair conventions are stable across the schema:

- **date_added + date_modified:** present on virtually every entity table. Some use `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` for `date_added`; others use `DATETIME` with a default of `'0000-00-00 00:00:00'`. The mix is a tell-tale of organic growth — different tables were added at different times.
- **status + sort_order:** present on most entity and lookup tables. The `status` semantics vary (see Section 4); `sort_order` is a manual display-ordering integer.
- **store_id (optional):** present when an entity is store-scoped. The convention is that `store_id = 0` means “all stores” or “default store”; non-zero values mean “this specific store only”.

7 Inferred HTTP Request Lifecycle

Although the PHP source is not in the repository, the OpenCart convention is rigid enough that we can reconstruct the request lifecycle with high confidence. The schema provides several direct signals: the `url_alias` table (the routing layer), the `stat` table (page-view logging), the `setting` table (configuration loading), the `language` table (translation loading), and the `widget` table (layout assembly).

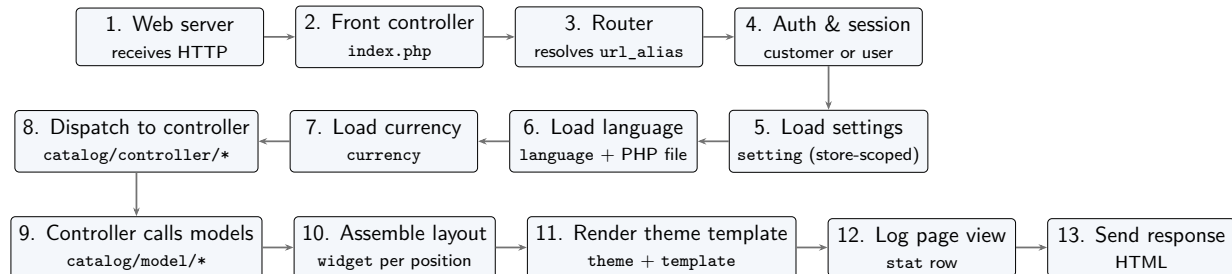


Figure 19: Inferred HTTP request lifecycle. The 13 phases reconstruct the standard OpenCart-derived front-controller pipeline. The schema touches the database at phases 3 (URL alias), 4 (session/customer), 5 (settings), 6 (language), 7 (currency), 8 (dispatch — the permission check on `user_group.permission`), 9 (model queries), 10 (widgets), 11 (theme), and 12 (stat).

7.1 Phase Notes

Phase 3 — Router: The router first checks `url_alias` for a slug matching the request path and the current `language_id`. If found, it reconstructs the internal `route=controller/method` string. If not found, it falls back to the query string `?route=...`. See Section 10 for the full algorithm.

Phase 4 — Auth: The session is loaded. If the customer is logged in, the `customer` row is fetched and cached. The visit counter (`customer.visits`) is incremented and the IP is updated. If the admin is logged in (admin route), the `user` row is fetched and the `user_group.permission` blob is decoded.

Phase 5 — Settings: All rows from `setting` with `store_id = 0` (defaults) and `store_id = current_store_id` (overrides) are loaded. The overrides win. The result is a flat `config` array.

Phase 6 — Language: The `language` row for the current language is loaded. The PHP file `catalog/language/<directory>/<filename>.php` is included; it populates a `$_` array of translation strings.

Phase 8 — Dispatch: The front controller instantiates the target `Controller` class and calls the target method. For admin routes, the user's `user_group.permission` is checked against the route. If the route is not in the `access` or `modify` list, the request is denied.

Phase 10 — Layout: The `widget` table is queried for widgets in the current store, position, landing page, and object type, sorted by `order`. Each widget's `extension` is loaded, its `settings` are decoded, and the widget controller is invoked to render into the layout position.

Phase 12 — Stat: A row is inserted into `stat` recording the page view, with the full HTTP context.

8 Inferred Customer Checkout Flow

8.1 Snapshot Writes at Confirmation

At order confirmation the application performs a multi-table, non-atomic write (MyISAM has no transactions). The order of operations is, by OpenCart convention:

1. Insert into `order` (snapshot of customer, address, shipping, payment, totals, currency, language).
2. For each cart line: insert into `order_product` (snapshot of name, model, price, tax, quantity), then insert into `order_option` for each selected option (snapshot of option name, value, price delta).
3. For each `order_total` line: insert into `order_total`.

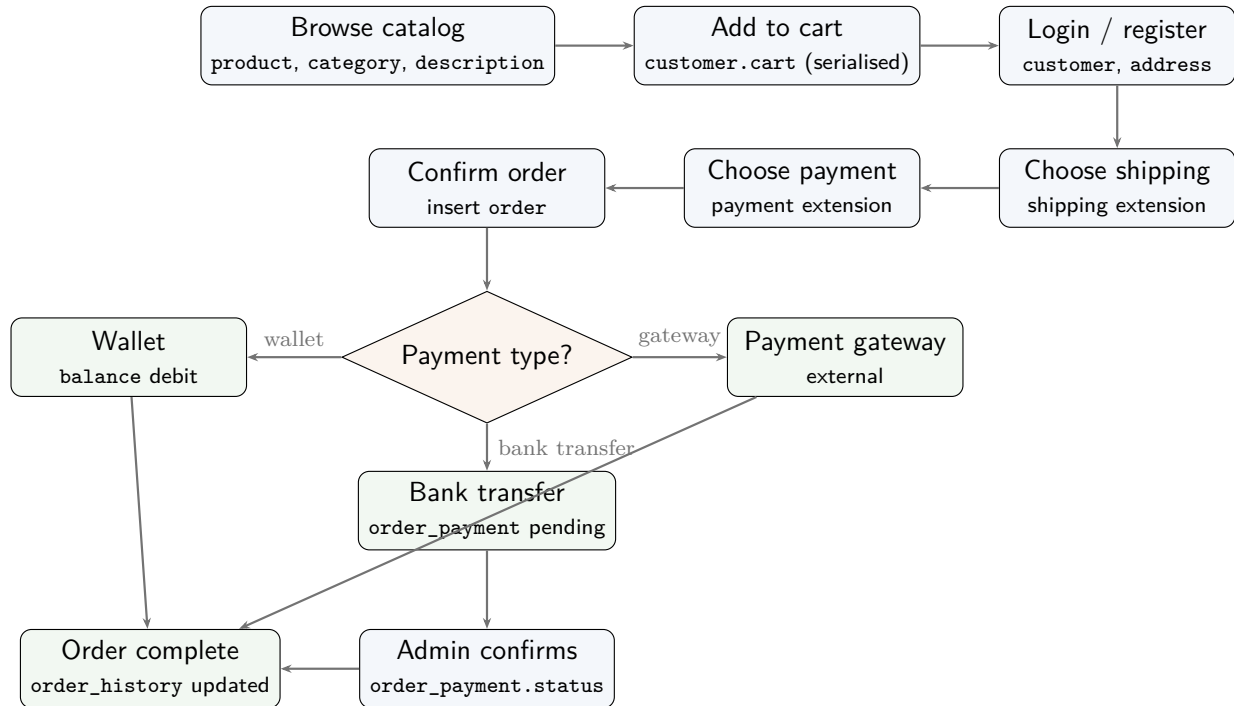


Figure 20: Inferred customer checkout flow. The cart is held in the `customer.cart` serialised blob (for logged-in customers) or in the PHP session (for guests). At confirmation, a row is inserted into `order` (with snapshot fields), `order_product`, `order_option`, `order_total`, and the first `order_history` row. The payment path branches into three: wallet debit (immediate), bank transfer (waits for admin confirmation), or external gateway (callback-driven). All three converge on a final `order_history` status update.

4. If the product is downloadable: insert into `order_download` for each download, with `remaining` set to the allowed download count.
5. If a coupon was used: insert into `coupon_history`.
6. Insert the first `order_history` row, with the initial `order_status_id` (typically “pending”).
7. If paying by wallet: insert a debit row into `balance`.
8. If paying by bank transfer: insert a pending row into `order_payment`.
9. Decrement stock: for each `order_product` line where `subtract = 1`, decrement `product.quantity` by the ordered quantity.
10. Optionally: insert into `task_queue` a confirmation-email task.

A crash between steps leaves the database in an inconsistent state. This is the inherent cost of the MyISAM choice (Section 2.5).

9 Inferred Task Scheduler Lifecycle

9.1 Real Cron vs. Pseudo-Cron

The schema alone cannot distinguish between a real `crontab` entry and a pseudo-cron triggered on each page load. Two arguments favour real cron:

- The marketing subsystem enqueues per-recipient email sends. A pseudo-cron on page loads would tie email throughput to site traffic, which is unreliable.
- The `task_queue.time_exec` column implies precise scheduling, which only a real cron can deliver.

We therefore infer a Unix `cron` entry running every minute, executing a PHP CLI script that performs the loop in Figure 21.

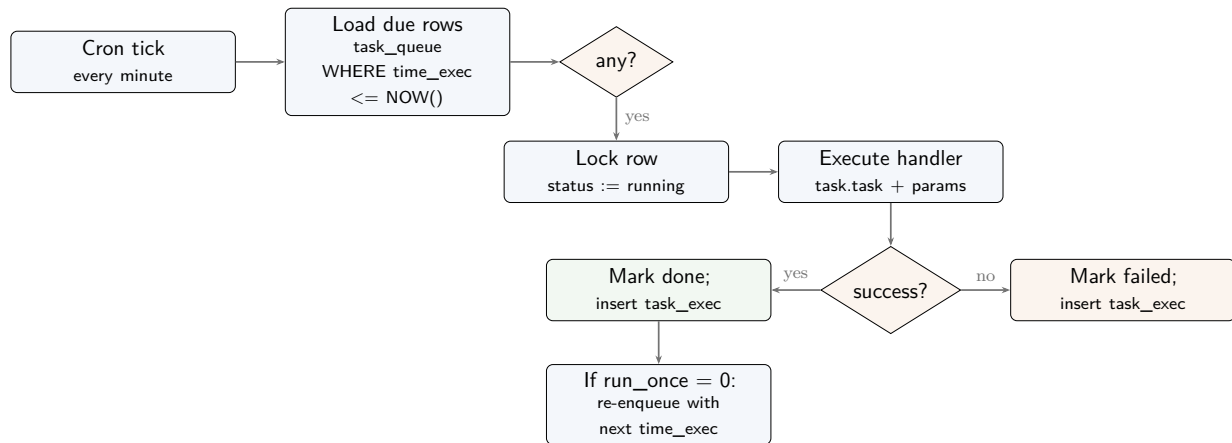


Figure 21: Inferred task-scheduler tick. The runner polls the `task_queue` table for due rows, locks one, executes its handler, writes a `task_exec` audit row, and either re-enqueues (recurring) or finalises (one-shot). The exact locking mechanism cannot be enforced by MyISAM, so the lock is a best-effort SQL update of the row's status from pending to running.

10 SEO and URL Routing

The `url_alias` table is the SEO-URL layer. Each row maps a slug to an internal query, polymorphically and per language. The inferred routing algorithm is:

1. **Inbound request** GET `/my-product` arrives.
2. The front controller looks up `url_alias` for `keyword = 'my-product'` AND `language_id = current_language_id`.
3. If found: the `query` column holds the internal route (e.g. `product_id=42`). The router translates this to `route=product/product&product_id=42` and dispatches.
4. If not found: the request falls through to the `?route=...` query-string convention, or returns a 404.

Outbound, when the application generates a link to a product, it looks up `url_alias` for `object_id = 42`, `object_type = 'product'`, `language_id = current` and, if found, emits `/my-product` instead of `?route=product/product&product_id=42`.

The two unique keys on `url_alias` enforce correctness:

- **UNIQUE (keyword, language_id):** a slug is unique within a language. Two slugs can collide across languages.
- **UNIQUE (object_id, language_id, object_type):** each object has at most one slug per language per type.

11 Security and Permissions Model

The security model has three layers:

11.1 Layer 1 — Customer Authentication

Customers authenticate with email + password (the `email` column is unique on `customer`). The `password` column is 100 characters wide, which fits modern hashes (bcrypt is 60 chars, Argon2id can be up to 97). On login, the application verifies the hash and updates `customer.ip` and `customer.visits`. Account activation is gated by `customer.activation_code` (emailed at signup) and `customer.approved` (admin-moderated). Banned customers are blocked via `customer.banned`.

11.2 Layer 2 — Admin Authentication

Admins authenticate with username + password against `user`. The `user.password` column is also 100 characters wide. After login, the admin's `user_group.permission` blob is decoded and used to gate every subsequent admin route.

11.3 Layer 3 — Route-Level Authorisation

The `user_group.permission` blob, by OpenCart convention, has the structure:

```
array(  
  'access' => array('catalog/product', 'catalog/category', ...),  
  'modify' => array('catalog/product', 'catalog/category', ...)  
)
```

The front controller, before dispatching an admin route, checks whether the route string is in the current user's `access` (for GET requests) or `modify` (for POST/PUT/DELETE requests) array. If not, the request is denied. This is a coarse, route-level permission model — finer-grained permissions (per-object, per-field) are not present in the schema.

11.4 Audit Trail

Every admin action is logged in `user_activity` with the polymorphic object reference, the event, the action, a description, the IP and the user-agent. This is the non-repudiation layer.

12 Data Integrity Observations

We repeat: this section is descriptive, not prescriptive. We list the integrity consequences of the schema's design choices without recommending changes.

12.1 Consequences of MyISAM

- No declared foreign keys. The 100+ “_id” columns are soft references. The application is solely responsible for referential integrity.
- No transactions. Multi-table writes (e.g. order insertion, Section 8) are not atomic.
- Table-level locking on writes. Concurrent writes to `stat` (one row per page view) can become a bottleneck on a busy site.
- Crashes can corrupt MyISAM tables; the schema's many `date_added` and `date_modified` columns help forensic recovery but cannot prevent it.

12.2 Consequences of the Polymorphic Pattern

- No cascade deletes. Deleting a `product` row leaves orphaned rows in `description`, `property`, `url_alias`, `review`, `stat`, `notification`, `user_activity`, and `object_to_store`. The application must clean these up explicitly, or accept the orphans.
- No cascade updates. Renaming the primary key of `object` (impossible in practice, but illustrative) would not propagate.
- No way to enforce that an (`object_id`, `object_type`) pair in `description` actually points to a real object. A typo in `object_type` silently creates an orphan.

12.3 Consequences of the “params” TEXT Escape Hatch

- The contents of `object.params`, `banner.params`, `widget.settings`, `extension.settings`, `task.params`, `task_queue.params`, `customer_group.params` are opaque to SQL.
- Schema migrations cannot change the structure of these blobs; the application must handle backward compatibility in PHP.

- A PHP `serialize()` / `unserialize()` round-trip can fail silently if the serialised object references a class that no longer exists or has changed.

12.4 Consequences of the Snapshot Pattern

- The `order` table is wide (43 columns) and deliberately denormalised. This is correct for legal fidelity but makes ad-hoc reporting harder.
- Renaming a product after an order does not affect the order's `order_product.name` snapshot, which is the intended behaviour.
- The `order_total` table stores the formatted `text` (e.g. “USD 120.00”) at order time. Currency reconfigurations do not retroactively change past orders.

13 Closing Observations

We close with a set of observations that do not fit anywhere else, and that the reader may find useful when interpreting the rest of the document. None of these are recommendations; they are simply features of the system that are easy to miss on a first reading.

13.1 The System Has Its Own Identity

The `template.for_nt_version` column, the table prefix `nts8sd4fd_`, the absence of OpenCart's `category_path` table, and the presence of the polymorphic `object` spine all indicate that “Necoyoad” is not a vanilla OpenCart install. It is either a fork of OpenCart or a separate system that borrowed OpenCart's conventions heavily. The author called the database `necoyoad_db`, suggesting the platform name itself is “Necoyoad”.

13.2 The Venezuelan Context is Visible

The `customer.rif` column, the `bank_account.rif` column, the `order_payment` subsystem (manual bank transfers as the primary payment method), the `campaign.message` SQL comment in Spanish (“Mensaje que sera traducido”), and the use of `utf8_spanish2_ci` collation on the `post` table's text columns all place the system's primary context in Venezuela. This is consistent with the author's GitHub username (`yosietserga` is a Latin-script rendering of a Spanish name).

13.3 The Schema Records Its Own Evolution

The inconsistent use of `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` versus `DATETIME DEFAULT '0000-00-00 00:00:00'`; the occasional typo (`data_added` on `task`); the mix of `latin1` default charsets with per-column `utf8_bin` overrides; the parallel existence of `product_to_category` (specific) and `object_to_category` (polymorphic) — all of these are the SQL-layer equivalent of growth rings. They let us read the schema as a palimpsest of features added at different times by different hands.

13.4 The Marketing Subsystem is the Most Sophisticated Part

After the catalog, the marketing subsystem (`newsletter`, `campaign`, `contact`, `contact_list`, `campaign_contact`, `campaign_link`, `campaign_stat`, `campaign_link_stat`, plus the `task_queue` integration) is the most behaviourally complex part of the system. It is essentially a self-hosted Mailchimp clone, embedded in the e-commerce platform, with full open and click tracking. This is non-trivial to build and suggests significant investment by the author.

13.5 The Audit Subsystem is Pervasive

The HTTP-context snapshot pattern (`server`, `session`, `request`, `ref`, `browser`, `browser_version`, `os`, `ip`, `date_added`) appears in five tables: `stat`, `search`, `user_activity`, `campaign_stat`, `campaign_link_stat`. This is a deliberate, schema-level commitment to per-event forensic logging. It is unusual to see this level of audit detail in a small e-commerce platform and indicates that the author treated analytics as a first-class concern.

14 Conclusion

The Necoyoad database dump, despite being a single 76-kilobyte SQL file, encodes a complete, working multi-store e-commerce and content platform. From the 87 tables alone we can reconstruct: the runtime stack (Apache or Nginx, PHP 8.1.2, MySQL 5.7.17, MyISAM); the architectural style (OpenCart-derived MVC with a polymorphic object spine); the four cross-cutting patterns (polymorphic associations, EAV-style descriptions, multi-store scoping, multi-language scoping); fifteen functional domains; three runtime lifecycles (HTTP request, customer checkout, task-scheduler tick); the SEO/URL routing layer; the three-layer security model; and the data-integrity consequences of every major design choice.

The reconstruction is necessarily incomplete in places — without the PHP source we cannot, for example, confirm the exact permission-blob format or the precise cron-scheduling algorithm — but the schema carries enough intent (in column names, SQL comments, unique-key choices, and engine/charset selections) that the gaps are smaller than one might expect. The eighteen TikZ diagrams and six inventory tables in this document are intended to serve as a faithful map of the system as it stood on the morning of 3 February 2023.

A Full Table Inventory

Table 6: All 87 tables in the `necoyoad_db` schema, with primary key and a one-line responsibility.

Table	PK	Responsibility
<code>address</code>	<code>address_id</code>	Customer address book entry.
<code>balance</code>	<code>balance_id</code>	Wallet ledger row (per customer, per currency).
<code>bank</code>	<code>bank_id</code>	Bank directory entry.
<code>bank_account</code>	<code>bank_account_id</code>	Merchant bank account (for receiving transfers).
<code>banner</code>	<code>banner_id</code>	Banner / slider definition.
<code>banner_item</code>	<code>banner_item_id</code>	Single slide / banner image.
<code>campaign</code>	<code>campaign_id</code>	Email-marketing campaign definition.
<code>campaign_contact</code>	<code>campaign_contact_id</code>	One recipient of one campaign.
<code>campaign_link</code>	<code>campaign_link_id</code>	Trackable link inside a campaign.
<code>campaign_link_stat</code>	<code>..._stat_id</code>	Link-click event.
<code>campaign_stat</code>	<code>campaign_stat_id</code>	Email-open event.
<code>category</code>	<code>category_id</code>	Polymorphic category node (tree via <code>parent_id</code>).
<code>contact</code>	<code>contact_id</code>	CRM contact (may map to a customer).
<code>contact_list</code>	<code>contact_list_id</code>	Named list of contacts.
<code>contact_to_list</code>	<code>contact_to_list_id</code>	Junction: contact ↔ list.
<code>country</code>	<code>country_id</code>	ISO country reference.
<code>coupon</code>	<code>coupon_id</code>	Discount coupon definition.
<code>coupon_category</code>	(coupon, cat)	Junction: coupon to category.
<code>coupon_history</code>	<code>coupon_history_id</code>	Coupon-redemption log.
<code>coupon_product</code>	<code>coupon_product_id</code>	Junction: coupon ↔ product.
<code>currency</code>	<code>currency_id</code>	Currency reference + exchange rate.
<code>customer</code>	<code>customer_id</code>	Storefront customer account.
<code>customer_group</code>	<code>customer_group_id</code>	Customer group (pricing/permission tier).
<code>description</code>	<code>description_id</code>	Polymorphic, localised rich-text content.
<code>download</code>	<code>download_id</code>	Downloadable file metadata.
<code>extension</code>	<code>extension_id</code>	Installed extension registry entry.
<code>geo_zone</code>	<code>geo_zone_id</code>	Grouping of zones for tax rules.
<code>language</code>	<code>language_id</code>	Supported language.
<code>length_class</code>	<code>length_class_id</code>	Length-unit reference.
<code>manufacturer</code>	<code>manufacturer_id</code>	Product manufacturer / brand.
<code>menu</code>	<code>menu_id</code>	Named menu (per store, per position).
<code>menu_link</code>	<code>menu_link_id</code>	Menu item (tree via <code>parent_id</code>).
<code>newsletter</code>	<code>newsletter_id</code>	Reusable email body (text + HTML).
<code>notification</code>	<code>notification_id</code>	Per-user notification.
<code>object</code>	<code>object_id</code>	Polymorphic entity spine.

Table	PK	Responsibility
object_to_category	..._category_id	Junction: any object to category.
object_to_store	..._store_id	Junction: any object to store.
order	order_id	Order header (snapshot).
order_download	order_download_id	Downloadable entitlement from an order line.
order_history	order_history_id	Order status-change log.
order_option	order_option_id	Snapshot of an option chosen on an order line.
order_payment	order_payment_id	Manual bank-transfer payment record.
order_product	order_product_id	Snapshot of an order line item.
order_total	order_total_id	One line in the cart total (sub-total, tax, shipping, ...).
post	post_id	CMS post (any <code>post_type</code>).
product	product_id	Catalog product.
product_attribute	product_attribute_id	Display-only product spec.
product_attribute_group	..._group_id	Group of product attributes.
product_discount	product_discount_id	Quantity-break discount.
product_image	product_image_id	Additional product image.
product_option	product_option_id	Orderable product option.
product_option_description	(id, lang_id)	Localised option name.
product_option_value	..._value_id	Orderable option value.
product_opt_val_desc	(id, lid)	Localised option-value name.
product_related	(id, rel_id)	Junction: related products.
product_special	product_special_id	Promotional price.
product_tags	(id, tag, lid)	Language-keyed tags.
product_to_category	(id, cat_id)	Junction: product to category.
product_to_download	(id, dl_id)	Junction: product to download.
product_to_zone	product_to_zone_id	Junction: product to zone.
product_type	product_type_id	Product-type lookup.
property	property_id	Polymorphic free-form key-value pair.
review	review_id	Polymorphic, threaded review with rating.
review_likes	review_likes_id	Like/dislike vote on a review.
search	search_id	Search-query log (with HTTP context).
setting	setting_id	Flat key-value config (store-scoped).
stat	stat_id	Page-view log (with HTTP context).
status	status_id	Normalised status lookup.
store	store_id	Store / tenant definition.
task	task_id	Scheduled-task template.
task_exec	task_exec_id	Task-execution audit row.
task_queue	task_queue_id	Materialised task instance (waiting to run).
tax_class	tax_class_id	Tax-class grouping.
tax_rate	tax_rate_id	Tax rate (per geo zone, per class).
template	template_id	Design-package definition.
theme	theme_id	Theme instance (per store, per user).
theme_style	theme_style_id	CSS-rule override for a theme.
url_alias	url_alias_id	SEO slug (polymorphic, language-scoped).
user	user_id	Admin user account.
user_activity	user_activity_id	Admin audit log (with HTTP context).
user_group	user_group_id	Admin user group + permissions blob.
warehouse_movement	movement_id	Inventory movement log (batch, expiry).
weight_class	weight_class_id	Weight-unit reference.
widget	widget_id	Positioned, configurable content block.
widget_landing_page	..._page_id	Widget-landing page junction.
zone	zone_id	State/province reference.
zone_to_geo_zone	zone_to_geo_zone_id	Junction: zone to geo zone.

B Polymorphic Association Inventory

Table 7: Every polymorphic (`object_id`, `object_type`) association in the schema, with the discriminator values inferable from the schema.

Table	Polymorphic columns	Inferable discriminator values
object	parent_id, parent_type	“product_category”, “post_category”, “customer_group” (per SQL comment)
object	object_type	“product”, “post”, “survey” (per SQL comment)
object	subtype	“digital”, “normal”, “auction” (per SQL comment)
object_to_category	object_id, object_type	any object type
object_to_store	object_id, object_type	any object type
description	object_id, object_type	any object type
property	object_id, object_type	any object type
url_alias	object_id, object_type	any object type
review	object_id, object_type	“product”, “post”
review_likes	object_id, object_type	inherits from <code>review</code>
stat	object_id, object_type	any object type
notification	object_id, object_type	any object type
user_activity	object_id, object_type	any object type
task	object_id, object_type	any object type
category	object_type	“product” (default), “post”, etc.
widget_landing_page	object_type	landing-page object type
campaign_stat	(implicit: campaign + contact)	marketing
campaign_link_stat	(implicit: campaign + contact + link)	marketing

C Common Column Conventions

Table 8: Recurring column conventions across the schema.

Convention	Where it appears
date_added + date_modified	virtually every entity table. Mixed <code>TIMESTAMP</code> and <code>DATETIME</code> defaults.
status int(1)	most entity tables: 0 = disabled, 1 = enabled.
status	object only: -1 = deleted, 0 = disabled, 1 = enabled.
enum(' -1', '0', '1', '')	
status varchar	status table: free-form code referenced by <code>object.status_id</code> .
sort_order int	display ordering on most lookup and entity tables.
store_id int	multi-store scope on <code>customer</code> , <code>menu</code> , <code>notification</code> , <code>product_attribute</code> , <code>review</code> , <code>search</code> , <code>stat</code> , <code>task</code> , <code>template</code> , <code>theme</code> , <code>widget</code> , <code>setting</code> .
language_id int	language scope on <code>description</code> , <code>url_alias</code> , <code>product_option_description</code> , <code>product_option_value_description</code> , <code>product_tags</code> , <code>order</code> .
params / settings TEXT	object, banner, customer_group, extension, widget, setting.value (sometimes), task, task_queue.
object_id + object_type	polymorphic spine; see Appendix B.
customer_id	soft FK to customer on address, balance, review, review_likes, order, order_payment, campaign_contact, notification, stat, user_activity, contact.
product_id	soft FK to product on product_to_category, product_to_download, product_to_zone, product_image, product_option, product_option_value, product_discount, product_special, product_related, product_tags, order_product, order_download, coupon_product, warehouse_movement.
order_id	soft FK to order on order_product, order_option, order_total, order_history, order_download, order_payment, coupon_history.

D Engine, Charset and Collation Summary

Table 9: Engine, charset and collation distribution across the 87 tables.

Attribute	Count	%	Notes
ENGINE=MyISAM	87	100%	No InnoDB, no transactions, no FKs.
DEFAULT CHARSET=latin1	70	80%	Most tables default to latin1; per-column overrides common.
DEFAULT CHARSET=utf8	12	14%	description, object, object_to_category, object_to_store, property, status, store, url_alias, extension, user, user_group, coupon_category.
DEFAULT CHARSET=utf8 COLLATE=utf8_bin	2	2%	extension, user_group.
DEFAULT CHARSET=utf8 COLLATE=utf8_spanish2_ci	1	1%	post (text columns only).
Other	2	2%	Mixed / explicit per-column.
Total	87	100%	

E Glossary

EAV (Entity-Attribute-Value) A database design pattern where entity attributes are stored as rows in a key-value table rather than as columns in the entity table. Necoyoad’s **description** and **property** tables are EAV variants specialised for localised text and free-form key-value metadata.

Front controller A single PHP entry point (**index.php**) that receives every HTTP request and dispatches it to the appropriate controller.

MyISAM A legacy MySQL storage engine. Faster than InnoDB for read-heavy workloads, but lacks transactions, foreign keys and crash recovery. Used by all 87 tables in this schema.

OpenCart An open-source PHP e-commerce framework first released in 2008. The Necoyoad schema borrows heavily from OpenCart’s table naming, column naming and conventions.

Polymorphic association A database design where a child table references one of several possible parent tables via a pair of columns (**object_id**, **object_type**). The database cannot enforce the reference; the application must.

RIF *Registro de Información Fiscal*, the Venezuelan tax identifier used for both individuals and companies. Appears in **customer.rif**, **bank_account.rif**, **order.rif**.

Snapshot pattern Storing a copy of foreign-keyed data (e.g. customer name, address) directly in the referencing row (e.g. **order**) at write time, so that the referencing row remains readable even if the source row later changes or is deleted.

Soft foreign key A column whose value is intended to reference a primary key in another table, but which is not declared as a **FOREIGN KEY** constraint. Necoyoad has no hard foreign keys anywhere.

Task scheduler A background process (cron or pseudo-cron) that polls the **task_queue** table and executes due rows. Used primarily by the marketing-campaign subsystem.